PROGRAMACIÓN ORIENTADA A OBJETOS · NRC 36862

LegalSync

Sistema de gestión de casos legales

Equipo CodeLex

Docente: PhD. Jorge Edison Lascano

Universidad de las Fuerzas Armadas ESPE

Primer parcial · Octubre 2026

EL PROBLEMA

Un estudio jurídico que trabaja con hojas de Excel

Plazos que se olvidan

Excel no avisa cuando se acerca una audiencia o vence un plazo judicial.

Información dispersa

Casos, documentos y evidencias están en archivos y carpetas separados.

Sin trazabilidad

No se sabe qué usuario modificó un expediente ni cuándo.

Sin control de acceso

Todos ven y editan todo, sin permisos por rol.

OBJETIVO Y ALCANCE

Centralizar los casos del estudio en una aplicación Java

Desarrollar, con programación orientada a objetos, un sistema de consola que registre expedientes, controle el acceso por rol, avise de los plazos y registre cada cambio.

- Registrar y actualizar casos con campos obligatorios
- Vincular documentos, evidencias y plazos
- Alertas de plazos al iniciar sesión
- Búsqueda por nombre del cliente
- Auditoría en log.txt, importación CSV y documentos TXT

Alcance del primer parcial

Aplicación de consola en NetBeans con persistencia en archivos TXT y CSV.

GUI, MVC y base de datos quedan para la Unidad 2.

PRODUCT BACKLOG

Los nueve requisitos están implementados

ID	Requisito	Dónde se cumple
PB-001	Roles y permisos	User.hasPermission() · Lawyer.restrictUserAccess()
PB-002	Registro de casos	CaseFile.register() · validateMandatoryFields()
PB-003	Referencia a documentos	Document guarda la ruta como texto
PB-004	Alertas de plazos	Deadline.calculateRemainingDays() · needsAlert()
PB-005	Búsqueda por cliente	SearchEngine.searchByClientName()
PB-006	Auditoría	AuditLog.registerEntry() → log.txt
PB-007	Login seguro	AuthenticationService.authenticate() · maskPassword()
PB-008	Importar CSV	CsvImporter.importCases() · parseLine()
PB-009	Generar documentos	DocumentGenerator.generate() → .txt

ESTRUCTURA DEL PROYECTO

Tres paquetes, cada uno con una responsabilidad

`model`

Las 16 clases del diagrama: usuarios, expedientes, plazos, búsqueda, importación y auditoría.

`utils`

FileManager y Validator: métodos static para leer archivos y validar datos.

`view`

LegalSyncApp con el main: login, menú y lectura por consola con Scanner.

Proyecto Maven en NetBeans · **Source Packages** con los 3 paquetes · **Test Packages** con 6 clases de prueba JUnit 5

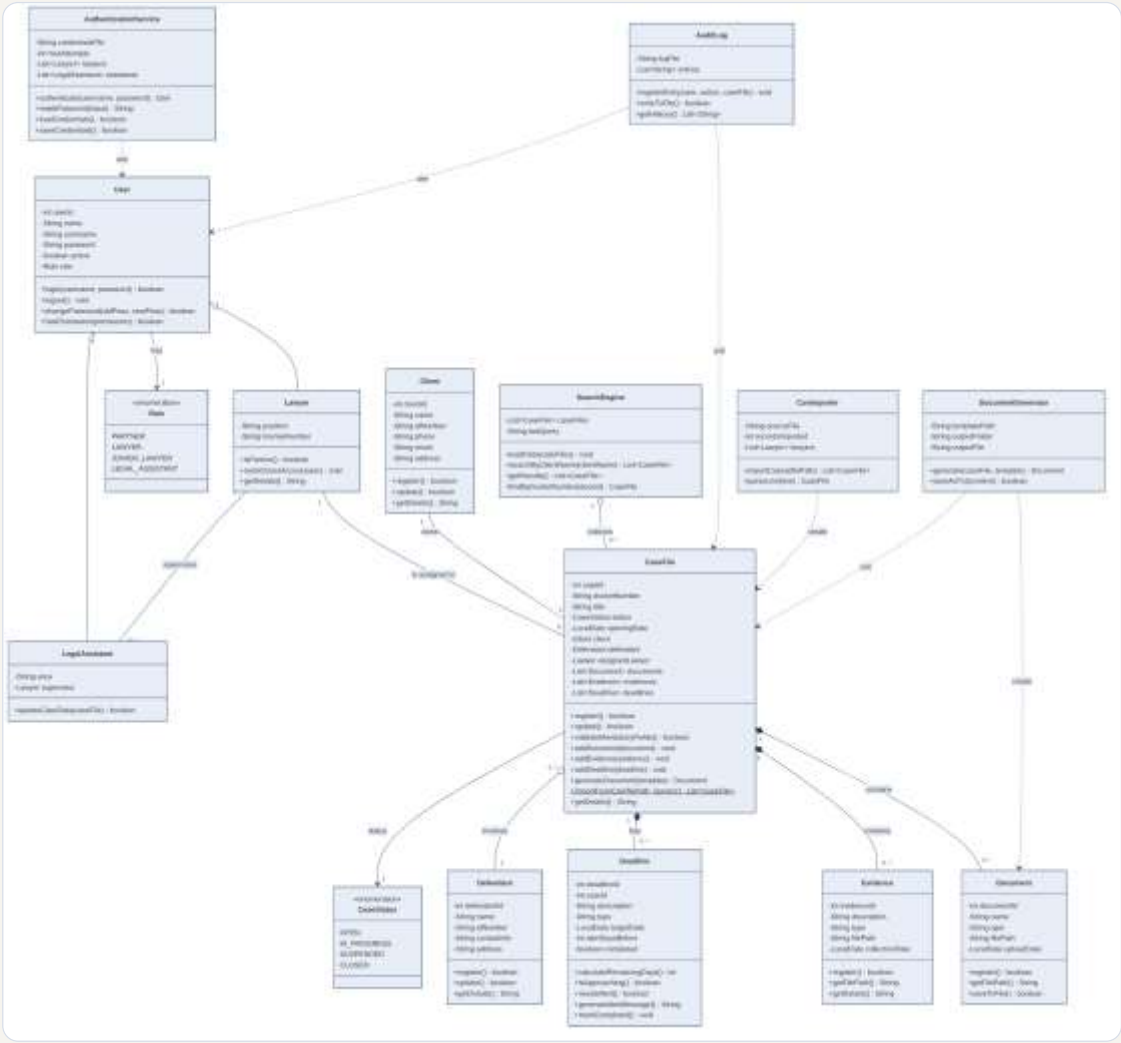
DISEÑO

Diagrama de clases final

14 clases y 2 enumeraciones en el paquete model.

CaseFile es la clase central: une al cliente, al demandado, al abogado y a sus documentos, evidencias y plazos.

Corresponde exactamente al código entregado.



HERENCIA

Un abogado y un asistente son usuarios

```
public class Lawyer extends User {
  private String position;
  private String licenseNumber;

  public Lawyer(...) {
    super(userId, name, username,
          password, role);
    ...
  }
}
```

Heredan de User

id, nombre, usuario, contraseña, estado y rol, más login(), changePassword() y hasPermission().

Lawyer agrega

position, licenseNumber, isPartner() y restrictUserAccess().

LegalAssistant agrega

area, supervisor y updateCaseData().

RELACIONES ENTRE CLASES

Cada relación del diagrama tiene su forma en Java

Asociación

CaseFile guarda a su **assignedLawyer** y a su **client**. Ambos existen aunque el caso se elimine.

Composición ◆

Las listas de **Document**, **Evidence** y **Deadline** se crean dentro del constructor de CaseFile: nacen y mueren con él.

Agregación ◇

SearchEngine indexa casos que ya existen afuera: **loadData(caseFiles)**.

Dependencia ⇢

AuditLog usa a User y CaseFile solo como parámetros de **registerEntry()**, sin guardarlos.

UNIDAD 1 APLICADA

Los temas del sílabo, dentro del código

Tema	Ejemplo en LegalSync
Encapsulamiento	Atributos private; la contraseña solo cambia con changePassword()
Constructores	CaseFile inicia en OPEN, con fecha de hoy y listas vacías
Getters y setters	Generados con NetBeans: getName(), setStatus(), isActive()
Enum	Role y CaseStatus: valores fijos, sin errores de escritura
Colecciones	List<CaseFile> con ArrayList, recorrida con for-each
Archivos	Scanner para leer, FileWriter para escribir TXT y CSV
Métodos static	Validator.isValidIdNumber(), FileManager.readLines()

ROLES Y PERMISOS · PB-001

Cada rol ve solo lo que le corresponde

Operación	Partner	Lawyer	Junior	Assistant
Registrar, actualizar, buscar casos	Sí	Sí	Sí	Sí
Plazos, evidencias, documentos	Sí	Sí	No	No
Importar desde CSV	Sí	No	No	Sí
Auditoría y usuarios	Sí	No	No	No

```
if (!currentUser.hasPermission("MANAGE_USERS")) → "Access denied"
```

FUNCIONAMIENTO · PB-004 Y PB-007

Al iniciar sesión, el sistema avisa los plazos

1. Carga usuarios y casos del CSV
2. Login con 3 intentos
3. Muestra las alertas de plazos
4. Menú de 13 opciones con permisos
5. Cada cambio queda en log.txt

```
Username: atorres
Password entered: **********
Welcome Ana Torres (PARTNER)

===== DEADLINE ALERTS =====
[!] UPCOMING: Preliminary hearing, 3 day(s) remaining
[!] OVERDUE: Appeal submission, 5 day(s) ago
[!] UPCOMING: Mediation meeting, 5 day(s) remaining
```

Los días se calculan con **ChronoUnit.DAYS.between(LocalDate.now(), targetDate)**; si el resultado es negativo, el plazo ya venció.

ARCHIVOS Y EXCEPCIONES

Un dato mal escrito no cierra el programa

Archivos en data/

credentials.txt: usuarios y roles

cases.csv: casos que se importan al iniciar

log.txt: auditoría, se escribe con FileWriter en modo append

output/: documentos legales generados

La línea 7 del CSV tiene una fecha inválida a propósito: se rechaza y se cargan los otros 5 casos.

try / catch específicos

NumberFormatException: letras donde va un número

DateTimeParseException: fecha como 2026-13-45

IllegalArgumentException: estado que no existe

FileNotFoundException: archivo inexistente

PRUEBAS UNITARIAS · JUNIT 5

76.92%

20 de 26 pruebas pasan, en 6 clases de prueba.

Cada clase tiene una prueba que falla a propósito, igual que divide(0, 0) en BasicOperations: las pruebas encontraron casos límite.

Prueba que falla	Por qué
testMaskPasswordNull	Contraseña null → NullPointerException
testGetDetails	Caso sin abogado → NullPointerException
testRegisterIdNumberWithDash	"171234567-8": el guion no es dígito
testCalculateRemainingDaysWithoutDate	Plazo sin fecha → NullPointerException
testSearchByClientNameEmpty	contains("") siempre es true → 3 casos
testLoginUppercaseUsername	equals() distingue mayúsculas

DEL DISEÑO AL CÓDIGO

Lo que mejoramos del diagrama original

Cambio	Razón
Rol LAWYER agregado a Role	El actor Lawyer existía en los casos de uso
status: String → CaseStatus	Usar el enum evita estados mal escritos
Lawyer y LegalAssistant extends User	La relación correcta es "es un"
Sin AuditEntry, Notification ni AlertService	Más simple: AuditLog guarda Strings y Deadline genera la alerta
importFromCsv(filePath, lawyers)	Necesita los abogados para asignarlos
Nombres corregidos	Authentification, Csvimporter, Defedant, Sutring

CONCLUSIONES

LegalSync cumple los 9 requisitos con los temas de la Unidad 1

Diseño

Herencia, asociación, agregación y composición aplicadas a un caso real.

Datos

ArrayList y archivos TXT/CSV, protegidos con excepciones.

Calidad

26 pruebas que detectaron 6 casos límite por mejorar.

¿Preguntas?

```
Demo: atorres / partner123
      cmena / lawyer123
```